

3-MONTH COMPREHENSIVE COURSE (BASIC TO ADVANCED)

Python Programming Syllabus

A structured journey through core runtime theory and modern application engineering

Duration: 12 Weeks (3 Months)	Daily Commitment: 2 – 3 Hours (Theory + Practice)	Target Audience: Students & Beginners	Language: English
---	---	---	-----------------------------

SYLLABUS ARCHITECTURE

- Month 1: Python Fundamentals
- Month 2: Intermediate Python
- Month 3: Advanced Python & Real-World Applications

MONTH 1: PYTHON FUNDAMENTALS

Week 1: Introduction to Python

CORE TOPICS

- **Environment Setup:** Python 3.12+ runtime installation; configuring VS Code or PyCharm IDE profiles
- **Data Foundations:** Working with Variables and basic memory allocations
- **Primitive Data Types:** Hand-on usage of integers (`int`), floats (`float`), strings (`str`), and booleans (`bool`)
- **Operators Engine:** Executing logic through Arithmetic, Comparison, and Logical expressions
- **I/O Subsystems:** Capturing console data with `input()` and formatting displays using `print()`

PRACTICAL LABORATORY

Write functional terminal programs for an algebraic calculator, a compound/simple interest accountant tool, and a dynamic Celsius/Fahrenheit temperature converter.

CURATED RESOURCES

- Official Python Documentation Standard (python.org references)
- YouTube Masterclass: "Python for Beginners" – freeCodeCamp (Comprehensive introductory block - first 4 hours)
- W3Schools Python Academy Modules (Chapters 1–5 foundational track)

Week 2: Control Structures

CORE TOPICS

- **Conditional Execution:** Structural flow control paths using ``if``, ``elif``, and fallback ``else`` blocks
- **Iteration Loops:** Processing data sets and indices with ``for`` loops and conditional state ``while`` loops
- **Loop Modifiers:** Controlling processing flows dynamically using ``break``, ``continue``, and empty code-placeholder ``pass`` operators
- **Multidimensional Control:** Designing and validating layered logic via Nested Loops structures

PRACTICAL LABORATORY

Engineer custom algorithms to evaluate prime numbers, execute custom structural terminal pattern designs, and build a random integer-guessing simulation game loop.

CURATED RESOURCES

- Textbook Reference: "Automate the Boring Stuff with Python" by Al Sweigart (Chapter 2 focusing on Flow Control)
- YouTube Lecture: "Python Control Flow & Core Conditionals" – Corey Schafer Series

Week 3: Functions & Modules

CORE TOPICS

- **Procedural Architecture:** Defining routines via functions, structuring parameters, and routing custom keyword/positional arguments
- **Data Pipeline:** Returning states with ``return``, and evaluating local namespace vs. global variable runtime context
- **Anonymous Logic:** Deploying inline functional code expressions via Lambda functions
- **Standard Library:** Importing and querying built-in functional packages (``math``, ``random``, ``datetime``)

PRACTICAL LABORATORY

Author clean, modular code repositories exposing decoupled functions for processing factorials, evaluating Fibonacci sequences, and exposing an advanced mathematical calculator engine.

CURATED RESOURCES

- Python.org Structural Guide (Definitive Functions Documentation)
- YouTube Guide: "Python Functions Deep Dive" – Tech With Tim Series

Week 4: Data Structures – Part 1

CORE TOPICS

- **String Manipulation:** Advanced index slicing methods, internal format methods, and f-string interpolation techniques
- **Lists:** Mutability concepts, multi-index slicing, element mutator methods, and declarative List Comprehensions
- **Tuples & Sets:** Immutable array constructs with tuples, and unique-value collection evaluation architectures using sets

PRACTICAL LABORATORY

Develop comprehensive structural manipulation programs optimizing array workflows, managing sorting transformations, and parsing/deduplicating collections seamlessly.

CURATED RESOURCES

- W3Schools Core Reference Library (Detailed Tracks for Strings, Lists, Tuples, and Sets)
- YouTube Code-along: "Python Data Structures: Working with Lists, Tuples, and Sets" – Corey Schafer

MONTH 2: INTERMEDIATE PYTHON

Week 5: Data Structures – Part 2

CORE TOPICS

- **Dictionaries:** Mapping unique keys to values, using lookup methods, and managing multi-tiered Nested Dictionaries
- **Advanced Comprehensions:** Writing declarative transformations via optimized List and Dictionary Comprehension constructs
- **Data Interoperability:** Parsing, converting, formatting, and unpacking structured web payloads using native `json` data serializations

PRACTICAL LABORATORY

Design and build an interactive terminal Contact Book or a mini Student Management System utility driven by nested object states and key-value storage maps.

CURATED RESOURCES

- Textbook Reference: "Automate the Boring Stuff with Python" (Chapter 5 on Dictionaries and Structuring Data)
- YouTube Lecture: "Python Dictionaries: Working with Key-Value Pairs" – Corey Schafer

Week 6: File Handling & Exception Handling

CORE TOPICS

- **File Input/Output:** Safely reading from and streaming data writes to standard flat Text and structured CSV documents
- **Resource Engineering:** Deploying safe workspace configurations using explicit `with` statement Context Managers`
- **Fault-Tolerant Code:** Catching and handling runtime exceptions using structured `try`, `except`, fallback `else`, and cleanup `finally` blocks`
- **Custom Error Contexts:** Creating class-level validation errors by inheriting from the root `Exception` class`

PRACTICAL LABORATORY

Program an atomic file tracker utility that reads and registers persistent student profiles inside CSV worksheets, including robust data safety checking to prevent program crashes during ingestion.

CURATED RESOURCES

- Real Python Technical Track: "Reading and Writing Files in Python"
- YouTube Guide: "Python File I/O & Exception Isolation" – Tech With Tim Track

Week 7: Object-Oriented Programming (OOP) – Part 1

CORE TOPICS

- **OOP Paradigm:** Understanding classes as structural templates and objects as distinct live application memory instances
- **State Constructors:** Constructing instance initialization routines using the special `__init__` method and configuring unique attributes`
- **Behavior Modeling:** Interacting with encapsulated internal data structures by engineering explicit Instance Methods
- **Inheritance Systems:** Extending data frameworks using Single and Multiple Inheritance architectures
- **Polymorphism:** Overriding methods across overlapping functional subclasses to build unified execution interfaces

PRACTICAL LABORATORY

Model an enterprise transactional Bank Account management script containing polymorphic logic for handling accounts, updating asset classes, and calculating ledger balances.

CURATED RESOURCES

- YouTube Master Series: "Python OOP Tutorials" – Corey Schafer (Comprehensive multi-part playlist)
- Python.org Core Guides (Official Object-Oriented paradigm manual)

Week 8: OOP – Part 2 & Modules

CORE TOPICS

- **Encapsulation:** Hiding internal module configurations and data properties using private naming conventions (`__variable`)
- **Advanced Method Types:** Configuring class configurations via `@classmethod` and separating stateless logical methods via `@staticmethod`
- **Metaprogramming Protocols:** Overloading internal operator states using Python Dunder/Magic methods (e.g., `__str__`, `__repr__`, `__add__`)
- **Architectural Decomposition:** Writing code across custom files, packaging modules, and setting up isolated library namespaces

PRACTICAL LABORATORY

Extend the foundational Banking System architecture by breaking parts into distinct package directories, controlling variable access with getters/setters, and using magic methods to easily summarize object details in text formats.

CURATED RESOURCES

- Real Python Complete Blueprint: "Object-Oriented Programming in Python"
- YouTube Reference: "Python Modules and Packages: How to Split Code Modules" – Corey Schafer

MONTH 3: ADVANCED PYTHON & REAL-WORLD APPLICATIONS

Week 9: Advanced Python Concepts

CORE TOPICS

- **Memory-Efficient Evaluation:** Custom stream parsing via custom Iterators and custom loop data evaluation utilizing memory-safe `yield` Generators
- **Metaprogramming Wrappers:** Modifying underlying function processes using reusable `@decorator` wrappers
- **Advanced Syntax:** Composing complex nested List, Set, and Dictionary operations containing conditional assignments
- **Temporal Calculations:** Parsing calendar structures and calculating exact timeline spans via the `datetime` and `timedelta` modules
- **String Matching Engines:** Tokenizing, extracting, parsing, and cleaning text structures via Regular Expressions (`re` module)

PRACTICAL LABORATORY

Engineer an enterprise-grade automated security system containing a cryptographically randomized password generator and a real-time regex-powered strict criteria validation engine.

CURATED RESOURCES

- YouTube Code-along: "Advanced Python Architecture Blocks" – ArjanCodes & mCoding playlists

Week 10: Libraries for Data & Automation

CORE TOPICS

- **Vectorized Computation:** High-performance numerical calculations and vector matrix array processing via NumPy
- **Data Science Wrangling:** Structuring multi-column relational information schemas using Pandas Series and DataFrames
- **Data Ingestion:** Standardizing multi-format operational data streams by parsing flat CSV and structured Excel workbooks
- **Visual Analysis:** Exporting graphical trend reports and distribution charts using Matplotlib and Seaborn engines

PRACTICAL LABORATORY

Ingest a public database file, handle empty missing records, aggregate conditional statistics, and render distribution charts to spot trends.

CURATED RESOURCES

- Kaggle Data Science Academy (Free hands-on tracks for NumPy and Pandas)
- YouTube Tutorial: "Pandas Data Analysis Suite Masterclass" – Corey Schafer Series

Week 11: Web Development & APIs

CORE TOPICS

- **Microframework Web Servers:** Routing client URLs, building HTTP view functions, and rendering dynamic markup via Flask
- **UI Templates:** Passing variables and structural conditions safely out into standard HTML frontends using the Jinja engine
- **REST Endpoint Engineering:** Structuring web service payloads, serving custom endpoints, and handling server status replies
- **External Client Ingestion:** Communicating with external web services and fetching content arrays using the modern Python `requests` library

PRACTICAL LABORATORY

Build a responsive, web-based Task Manager application or an interactive Weather dashboard app that queries real-time global weather conditions from public REST APIs.

CURATED RESOURCES

- Flask Core Engineering Library (Official Quickstart Reference Docs)
- YouTube Masterclass: "Flask Web Development from Scratch" – Tech With Tim Course

Week 12: Projects, Testing & Best Practices

CORE TECHNICAL CAPSTONES (SELECT 2 DELIVERABLES)

- **Full-Stack Application:** Formulate a multi-user, CRUD-compliant Task Engine or custom Blog platform via Flask servers
- **Analytical Dashboard:** Build a automated dataset cleaning and visualization dashboard app using Pandas and Matplotlib data loops
- **Operating System Scripting:** Program an autonomous directory housekeeper automation script or a target multi-page web scraper

PRODUCTION GUIDELINES

- **Quality Engineering:** Designing assertion suites and executing automated test checks using the ``pytest`` testing ecosystem
- **Dependency isolation:** Setting up local virtual sandboxes (``venv``), package management with ``pip``, and pinning configurations inside ``requirements.txt`` maps
- **Style Compliance:** Refactoring syntax trees to align with Python PEP 8 stylistic standards and optimize operational speed
- **Distributed Version Control:** Establishing local git repositories, managing staging commits, and hosting source control pipelines on GitHub

CURATED RESOURCES

- Textbook Study: "Python Crash Course" by Eric Matthes (Project Blueprint Chapters)
- YouTube Portfolio Series: "Python Production Projects for Portfolio Optimization" – Tech With Tim
- Official ``pytest`` Automated Software Test Framework Documentation